

MEAN,MEDIAN,MODE(MEASURES OF CENTRAL TENDENCY)

```
In [1]: #MEAN(Average)
marks=[40,50,38,55,79]
mean = sum(marks)/len(marks)
print("Mean:",mean)
```

Mean: 52.4

```
In [6]: #MEDIAN(MIDDLE VALUE)
import statistics
marks=[40,50,66,73,87]
median = statistics.median(marks)
print("Median:",median)
```

Median: 66

```
In [13]: #MODE
import statistics
data=[10,20,20,30]
mode = statistics.mode(data)
print("Mode:",mode)
```

Mode: 20

```
In [17]: import pandas as pd
df=pd.DataFrame({
    'Marks':[40,50,60,70,53]
})
print("Mean:")
print(df["Marks"].mean())
print("\nMedian:")
print(df["Marks"].median())
print("\nMode:")
print(df["Marks"].mode())
```

Mean:

54.6

Median:

53.0

Mode:

0 40

1 50

2 53

3 60

4 70

Name: Marks, dtype: int64

VARIANCE AND STD - MEASURE OF SPREAD

```
In [19]: #Variance
import numpy as np
data = [10,12,14,16,18]
print(np.mean(data))

variance=np.var(data)
print("Variance:",variance)
```

14.0

Variance: 8.0

```
In [22]: #STANDARD DEVIATION
#small std - Data points are close to mean
#Large std - data points are far from mean.
#Variance
import numpy as np
data = [10,12,14,16,18]

variance=np.var(data)
std = np.var(data)

print("Variance:",variance)
print("Std:",std)
```

Variance: 8.0

Std: 8.0

IQR

```
In [27]: #IQR
#Less affected by outliers than variance and std.
#IQR = Q3-Q1
import numpy as np
data = [5,10,15,20,25,30,35,40]
Q1 = np.percentile(data,25)
Q3 = np.percentile(data,75)
IQR = Q3-Q1
print("Q1:",Q1)
print("Q3:",Q3)
print("IQR:",IQR)

#Q1-1.5*IQR - Lower limit
#Q3+1.5*IQR - Upper limit
```

Q1: 13.75

Q3: 31.25

IQR: 17.5

```
In [31]: import numpy as np
data = [5,10,15,20,25,30,35,40,100]
Q1 = np.percentile(data,25)
Q3 = np.percentile(data,75)
IQR = Q3-Q1
lower=Q1-1.5*IQR
upper = Q3+1.5*IQR
print(lower)
print(upper)
outliers=[x for x in data if x < lower or x > upper]
print("Outliers:",outliers)
```

-15.0

65.0

Outliers: [100]

PROBABILITY BASICS

```
In [34]: #probability is the measure of how likely an event is to occur
# 0 - impossible
# 1 - Certain Event
sample_space={1,2,3,4,5,6}
event_A = {2,4,6}#Even Numbers
print("Sample_space(5):",sample_space)
print("Event A:",event_A)

#2.Probability p(A)
P_A = len(event_A)/len(sample_space)
print("/nprobability of Event A")
print("P(A)=",P_A)
```

```
Sample_space(5): {1, 2, 3, 4, 5, 6}
Event A: {2, 4, 6}
/nprobability of Event A
P(A)= 0.5
```

```
In [36]: #3.Conditional Probability
#p(a/b)
event_B = {4,5,6}#Number Greater than 3
intersection = event_A.intersection(event_B)
P_A_given_B = len(intersection)/len(event_B)
print("/nEvent B:",event_B)
print("A intersect b :",intersection)
print("P(A|B)=",P_A_given_B)
```

```
/nEvent B: {4, 5, 6}
A intersect b : {4, 6}
P(A|B)= 0.6666666666666666
```

```
In [41]: #4.Bayes Theorm
#Disease Prevaience - 1%
#Test sensitivity -99%
#Positive test probability - 5%
P_Disease = 0.01
P_Positive_given_Disease = 0.99
P_Positive = 0.05
P_Disease_given_Positive =(
```

```

    P_Positive_given_Disease*P_Disease)/P_Positive
print("\nBayes' Theorm Example")
print("P(Disease)=",P_Disease)
print("P(Positive|Disease)=",P_Positive_given_Disease)
print("P(Positive)=",P_Positive)
print(
    "P(Disease|Positive)=",
    round(P_Disease_given_Positive,4)
)
print(
    "Percentage=",
    round(P_Disease_given_Positive*100,2),"%")

```

Bayes' Theorm Example
 P(Disease)= 0.01
 P(Positive|Disease)= 0.99
 P(Positive)= 0.05
 P(Disease|Positive)= 0.198
 Percentage= 19.8 %

```

In [42]: #Correlation
import pandas as pd
df=pd.DataFrame({
    'Hours':[1,2,3,4,5],
    'Marks':[20,40,50,79,90]})
corr = df['Hours'].corr(df['Marks'])
print("Pearson Correlation:",corr)

#Close to +1
#Strong Positive Relation

```

Pearson Correlation: 0.9894487353089338

```

In [45]: #Spearman Correlation - monotonic
#Relationship is not perfectly Linear.
import pandas as pd
df = pd.DataFrame({
    'Rank_Study':[1,2,3,4,5],
    'Rank_Marks':[2,1,3,5,4]
})
corr = df['Rank_Study'].corr(df['Rank_Marks'],method='spearman')
print("Spearman Correlation:",corr)

```

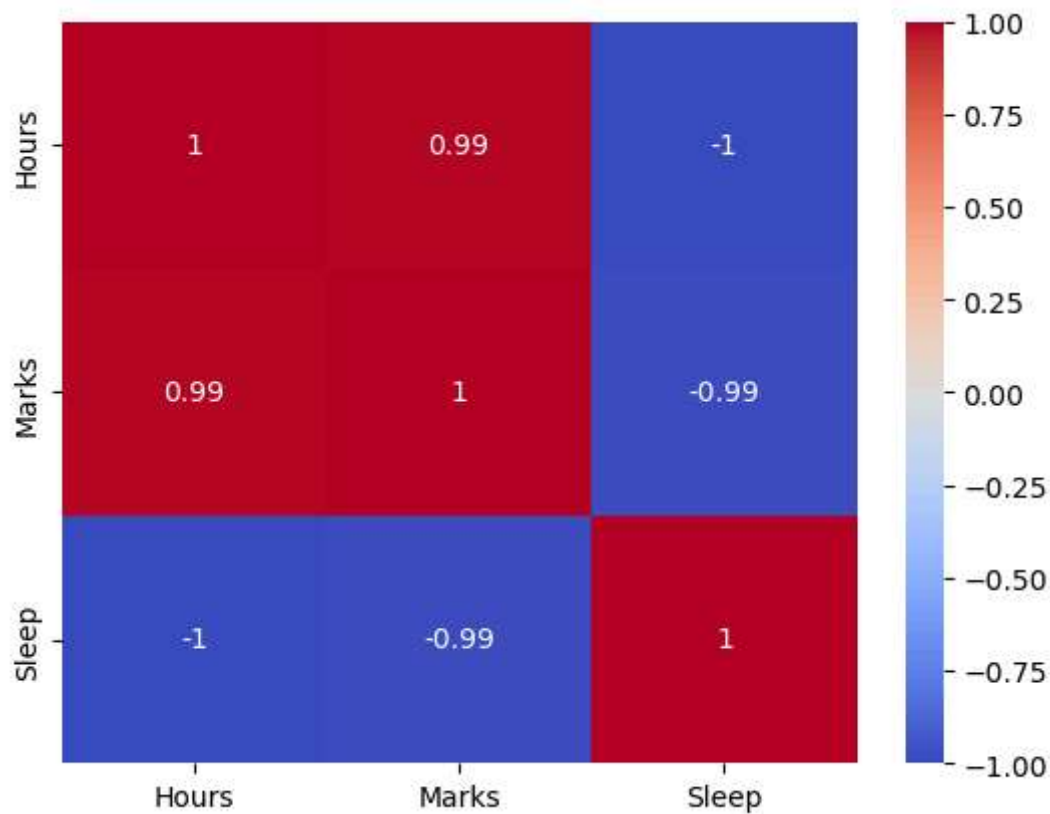
Spearman Correlation: 0.7999999999999999

```
In [47]: #Correlation matrix
import pandas as pd
df=pd.DataFrame({
    'Hours':[1,2,3,4,5],
    'Marks':[20,40,50,60,70],
    'Sleep':[8,7,6,5,4]
})
print(df.corr())
```

	Hours	Marks	Sleep
Hours	1.000000	0.986394	-1.000000
Marks	0.986394	1.000000	-0.986394
Sleep	-1.000000	-0.986394	1.000000

```
In [61]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df=pd.DataFrame({
    'Hours':[1,2,3,4,5],
    'Marks':[20,40,50,60,70],
    'Sleep':[8,7,6,5,4]
})
sns.heatmap(df.corr(),annot=True,cmap='coolwarm')
plt.show()
#Column cmap Values
#CoolWarm
#Viridis
#YlGnBu
#blues
#red
#greens
#magma
#plasma
```



Correlation vs Causation

```
In [7]: #Causation
#Causation means that one variable directly causes a change in another variable.

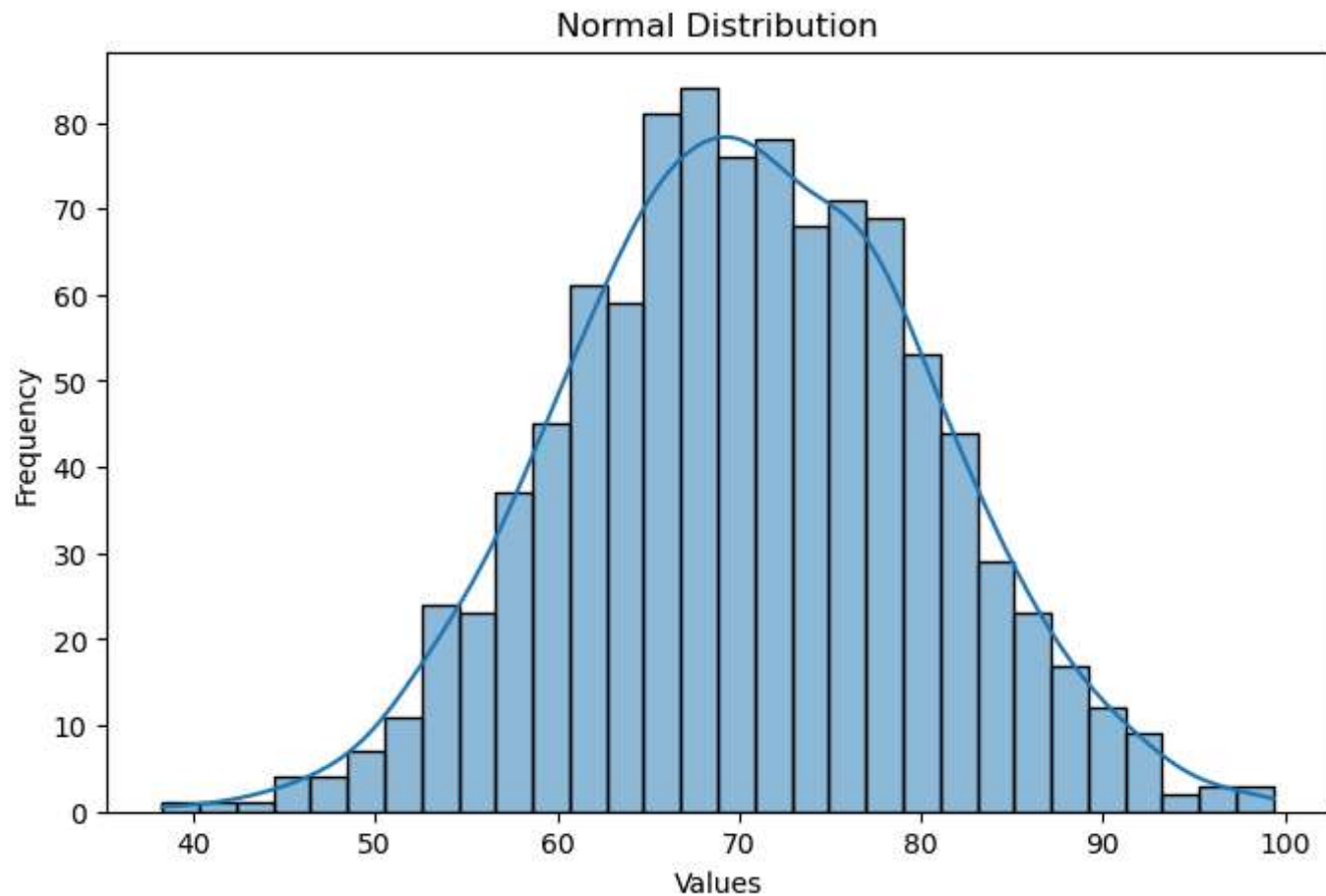
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
#Generate normally distributed data
data = np.random.normal(
    loc=70, #Mean
    scale=10, #std
    size=1000 #Number of values
)
#calculate mean and std
```

```
mean = np.mean(data)
std=np.std(data)
print("Mean:",round(mean,2))
print("Standard Deviation:",round(std,2))

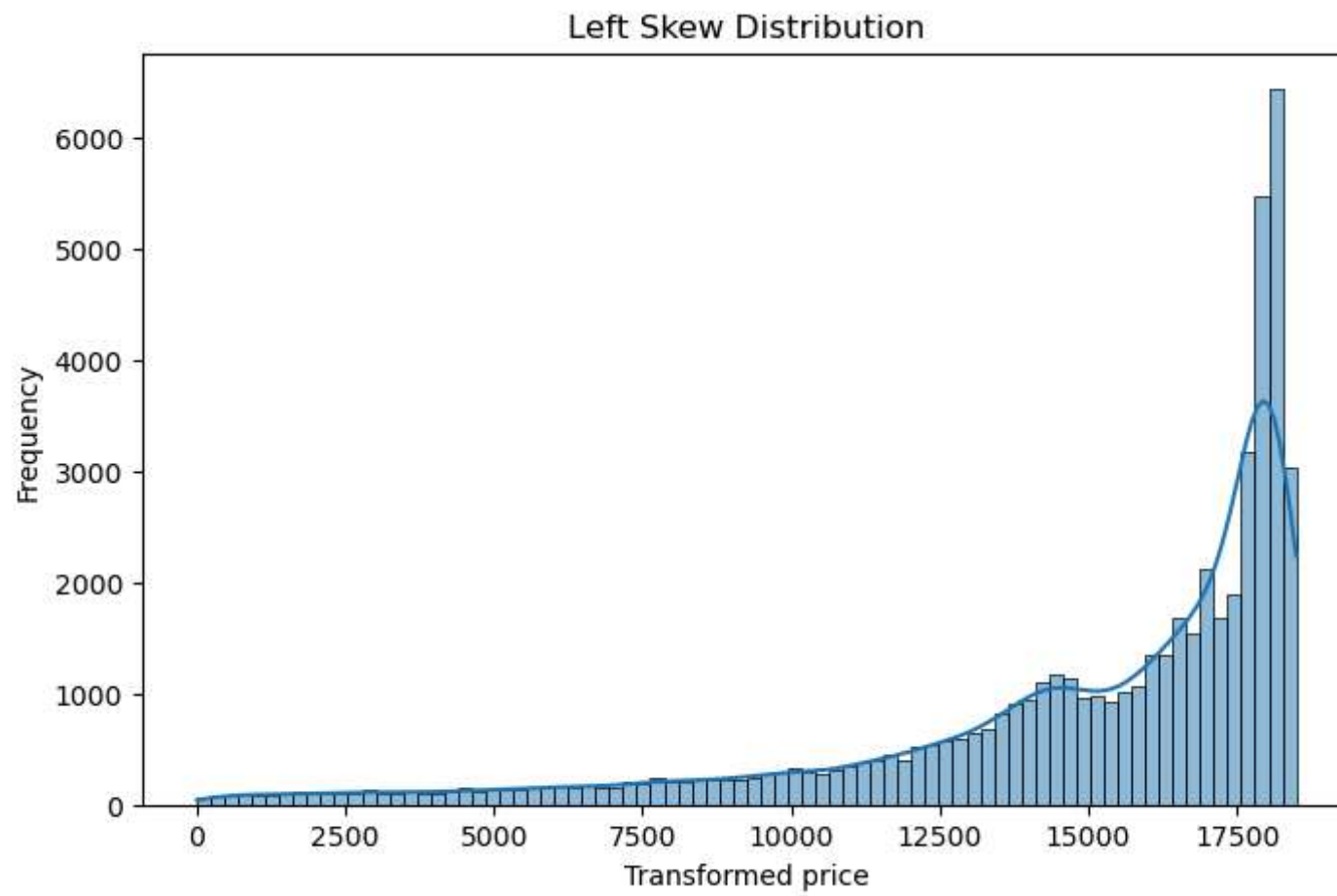
#Plot Histogram
plt.figure(figsize=(8,5))
sns.histplot(data,bins=30,kde=True)
plt.title("Normal Distribution")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.show()
```

Mean: 70.51

Standard Deviation: 9.78



```
In [15]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
df=sns.load_dataset("diamonds")
left_skew = df['price'].max()-df['price']
plt.figure(figsize=(8,5))
sns.histplot(left_skew,kde=True)
plt.title("Left Skew Distribution")
plt.xlabel("Transformed price")
plt.ylabel("Frequency")
plt.show()
print("Skewness:",left_skew.skew())
```



Skewness: -1.6183952833835282

In []: